Carl Ianne’s Spending Tracker: Guide for Students and Professors

Guide version 1.0 · October 30, 2026 · Describes project version 1.1.0 (about 1,580 lines of application JavaScript and 227 lines of CSS)

1. About this guide

Purpose. This guide explains what the Spending Tracker does, how it was built, why it was built that way, how its main features are technically possible, and where it falls short. It is written for two readers at once: a first-year BSIT student who wants to learn from a real working program, and a professor who wants to judge the design and the evidence behind it.

Who made what. The requirements and design decisions came from Carl Ianne, a first-year BSIT student. All of the code, the tests, the flowchart and this guide were written by Claude, an AI assistant made by Anthropic, during a chat conversation. Carl directed the work and will test the app on his own devices. This matters for how you read the guide: the program has been checked mostly by automated tests, not by weeks of real-world use.

How to read it.

- Students: read sections 2 to 4, then keep the code open beside sections 7 and 8. Try the exercises in section 12.
- Professors: sections 4, 5, 9, 10 and 11 cover the design decisions, the process, the evidence and the limitations. Section 12 has discussion questions.

How claims are labeled. To keep this guide objective, important claims carry one of three labels.

- **Measured**: we ran it and saw the result.
- **Reasoned**: it follows from the code or from a published standard, but we did not run it.
- **Unverified**: it has not been tested.

Status at a glance

Area	Status	Label
Logic, validation and security rules (Node 22 test run)	20 test groups pass	Measured
Real-browser behavior (Playwright 1.56, bundled Chromium, desktop-sized and phone-sized screens)	38 tests pass (19 tests, each run at two screen sizes)	Measured
Offline mode, install-button logic, backups and exports	Covered by those browser tests	Measured
Text contrast (WCAG AA, 4.5:1) in both themes	Passes in a unit check and in a real-browser check	Measured
ESLint rules	Configuration written but never run (the build environment had no network). A TypeScript-compiler check found 0 unused variables or imports and 0 undefined names	Partly measured
Deployment on Vercel	Configuration written, not yet deployed by us	Unverified
Safari, iPhone, Firefox, real Android hardware	Not tested	Unverified
Long-term real use (weeks of daily use)	Not yet done	Unverified
Independent human code review or security audit	None so far	Unverified

2. What the app does

The Spending Tracker helps a student plan an allowance. You record money received and money spent, and the app shows what is left and whether spending is within a budget. Everything runs in the browser. There is no server and no account.

2.1 Features

Feature	What it does
Add, edit, delete records	The + button opens a form: Spent or Received, amount, category, date, and an optional note (up to 80 characters)
Money left	Everything received minus everything spent up to the end of the viewed month, so unspent money carries over
Monthly budget limit	A yellow warning at 80% used, and a red alert when spending goes over the limit
Category limits	An optional limit per spending category; a category over its limit raises its own alert
Savings goal	A name and a target amount; a progress bar compares the target with money left
Projection	Current month only: average spending per day so far, projected spending for the whole month, and projected money left at month end
Charts	Donut chart of spending by category, weekly bar chart (Monday to Sunday), and a side-by-side comparison of two months
Themes	Dark (default: dark gray with forest green) and light (white with Google-style blue, red, yellow and green blocks)
Backup	JSON export, and JSON import with Merge or Replace. A reminder appears after 14 days without a backup
Spreadsheet export	CSV and a real Excel (.xlsx) file. These are export-only by design
Install and offline	An Install button appears when the browser allows it. After the first visit the app works with no internet

Feature	What it does
Currency	The symbol can be ₱, $, €, £ or ¥. It is a label only; there is no currency conversion

2.2 Quick start

1. Open the app and tap the + button.
2. Choose Received, enter your allowance, and save.
3. Add spending as you go (Spent, amount, category).
4. Open Budget and set a monthly limit and, if you like, a savings goal.
5. Check Home for money left and alerts, and Reports for the weekly chart and the month comparison.
6. Every week or two, open Data and export a JSON backup.

2.3 Rules the app follows

- Amounts are stored as positive numbers rounded to 2 decimals. The type (Received or Spent) supplies the sign.
- Dates are text like 2026-10-28 and months are text like 2026-10. Weeks run Monday to Sunday.
- A record with an unknown category is saved as Other rather than rejected.

3. How the program works

Figure 1 shows the whole program on one page. Part 1 runs once when the app opens. Part 2 repeats for every tap, edit, import or export. If the figure does not appear in your copy of this guide, open docs/flowchart.png in the project.

3.1 Part 1: starting the app

1. The browser loads index.html, styles.css and the JavaScript files. A security header (Content-Security-Policy) tells the browser to accept files only from the app's own site.
2. main.js starts. It connects the Install button and the + form, and builds the table that maps button names to code.

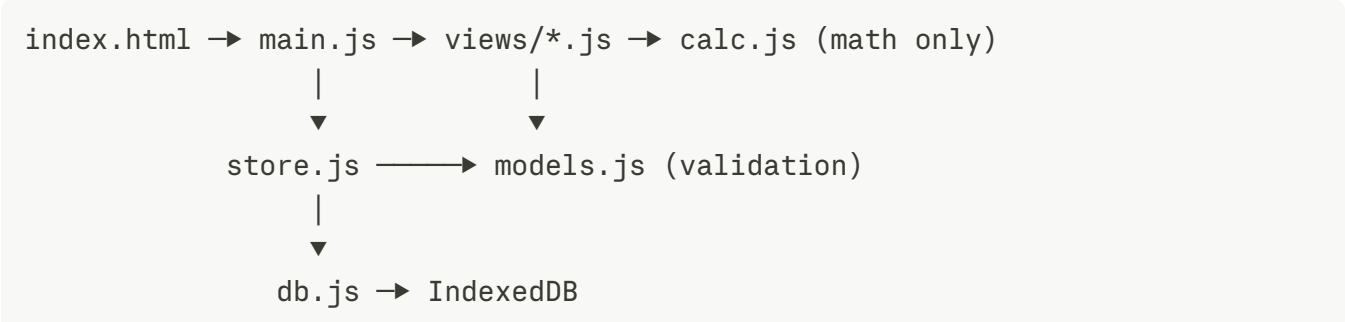
3. store.js asks db.js to read all records and settings from IndexedDB, the database built into the browser. models.js checks every record, and broken ones are dropped. If the database cannot open (private browsing, or another tab with an older version still open), the app shows a message and stops. Otherwise main.js asks the browser not to delete the data.
4. router.js reads the address (for example #/budget) and picks a view. The view builds the screen using numbers from calc.js. Last, main.js registers the service worker (sw.js) so the app works offline.

3.2 Part 2: every user action

5. The user does something: taps +, saves a budget, imports a file.
6. main.js finds the code to run from the button's name. An unknown name prints a console warning instead of failing silently.
7. models.js validates the input. If it is invalid, an error message appears and nothing is saved. If it is valid, store.js saves it through db.js and updates its own copy of the data.
8. store.js announces that data changed, and the current view runs again with the new numbers. Then the app waits for the next action.

3.3 Three ideas behind the design

- **Single source of truth.** All current data lives in one object inside store.js. Screens never keep their own copy of money data.
- **One-way data flow.** Data travels from the database to the store to the screen. Actions travel from the screen to the store to the database. A screen never edits the database directly.
- **Draw from data.** Each screen is a function that turns data into HTML. After every change the function runs again, so a screen cannot show stale numbers. The cost is that the whole screen is rebuilt each time (see section 11).



4. Technology choices and why

Choice	Why it was chosen	Main alternative	Trade-off
Plain HTML, CSS and JavaScript with ES modules; no framework and no build step	Easiest to read and learn; deploys as plain files; nothing to compile	React, Vue, Svelte	Screens are built from template strings by hand, with no component system or type checking. A framework scales better for large apps
IndexedDB	Required by the brief; stores structured records; asynchronous, so large reads do not freeze the page	localStorage	The raw IndexedDB API is verbose, so db.js wraps it. localStorage is simpler but stores only strings and blocks the page while it reads
PWA (manifest plus service worker)	Makes the app installable and usable offline	An ordinary website	More moving parts: caching rules and update behavior
GitHub plus Vercel	Free static hosting with automatic HTTPS (service workers need HTTPS, except on localhost) and automatic redeploy on every git push	Netlify, GitHub Pages	Tied to one vendor’s settings file (vercel.json)
Hand-written SVG charts	No library to download or update; works offline	Chart.js	Only the chart types we need; no animation
Hand-written .xlsx writer	An .xlsx file is a ZIP of XML files, so it can be written directly without a library	SheetJS	One sheet, no styling, dates written as text

Choice	Why it was chosen	Main alternative	Trade-off
CSS variables for themes	A theme is one attribute on the html element; each color is defined once	Two separate stylesheets	Readable contrast must be checked in both themes (done by tests)
Node test runner, Playwright, ESLint, GitHub Actions	Catch mistakes automatically at three levels: logic, real browser, code style	Manual testing only	More files to maintain; ESLint has not been run yet

5. How and where it was built

5.1 Where

- **Authoring.** In a conversation with Claude, an AI assistant by Anthropic, in the Claude web app. The code was written as files in the assistant's sandboxed Linux environment (Ubuntu, Node.js 22).
- **Testing.** Also inside that sandbox: Node's built-in runner for logic, and Playwright 1.56 with a bundled headless Chromium for the browser tests.
- **Runtime.** On the user's own device, inside the browser. No server code exists. The data never leaves the device unless the user exports a file.
- **Hosting (planned).** The project folder goes to GitHub, and Vercel connects to the repository and serves it. We have not performed this step.

5.2 Requirements from the brief

A minimal but good-looking interface in dark (default) and light themes; an installable PWA with a clear install button; deployment through Vercel connected to GitHub; IndexedDB storage; recording allowance and spending; remaining budget and limit warnings; categories, weeks, savings targets and projected balance; saved history, period comparison and charts; JSON import and export; CSV and Excel export without import; modular files with beginner-friendly comments; and a flowchart.

5.3 Process: five rounds

Round	Goal	Result
1. Initial build	A modular app that meets the brief	About 25 source files and a first flowchart
2. "Spaghetti" review	Check structure and readability	Measured the import graph: no circular imports, and the pure logic files never import UI or database code. Nine refactors, such as splitting a 68-line function into small ones
3. Production-style review	Hunt for hidden risks	Found issues in security, data safety and recovery (see 5.4), fixed with new tests
4. Browser tests, CI, contrast, lint	Prove behavior in a real browser	38 browser tests, a CI workflow, contrast fixes and an ESLint configuration. Found a real phone layout bug
5. Flowchart and this guide	Document the program for learners and reviewers	An updated flowchart and this document

5.4 Problems found and fixed

Problem	How it was found	Fix
"Replace" import erased data first and wrote the new data second, so a failure in between could lose everything	Code review	Both steps now run in one all-or-nothing database transaction
Two month input boxes on the Reports screen were not validated or escaped	Code review, then a test using a malicious value	Months must match YYYY-MM, and the value is escaped when drawn

Problem	How it was found	Fix
An imported record could carry a creation time of Infinity, which breaks sorting	Test	The time must be a finite number above 0
No request to keep the data, and no backup reminder	Review	The app calls persist() and shows a reminder after 14 days
A failed database open was remembered forever, and an upgrade blocked by an old tab froze the app silently	Review, then a fake-database test	The failure is forgotten so the next call retries, and a blocked upgrade shows a clear message
Light-mode text contrast was too low: 3.06:1 for white on the green block, 3.92:1 on red, 3.56:1 on blue (WCAG AA needs 4.5:1)	Measured	Darker Google shades for blocks that carry text
On a 412 px phone, the Reports page rendered 563 px wide and scrolled sideways	Real-browser test	A CSS grid column of minmax(0, 1fr) so wide charts scroll inside their card
A helper named alert hid the browser’s built-in alert	Review	Renamed, and an ESLint rule now forbids hiding built-in names
The offline test reloaded before the service worker had taken control	Real-browser test	A test bug, not an app bug: the test now waits for control

5.5 An honest limit of this process

The tests and the code were written by the same AI, so both can share the same blind spot. Three things reduce that risk. First, mutation checks: we deliberately broke the code in 10 places and confirmed that a test failed. Nine were caught at once, and the missed one (a missing escape) led to a stronger test, after which two further variants of that break were caught too. Second, a real browser rather than simulated pages. Third, measured numbers instead of opinions. None of this replaces review by an independent person, and section 11 lists that as a limitation.

6. Project structure and rules

```
spending-tracker/
├── index.html                the page shell: menu, + button, pop-up form
├── manifest.webmanifest      PWA information: name, icons, colors
├── sw.js                     service worker: offline cache
├── vercel.json               hosting settings and security headers
├── css/styles.css            all looks; themes are CSS variables
├── js/
|   ├── config.js             categories, currencies, defaults
|   ├── models.js             record shape and validation
|   ├── calc.js               all the math (pure functions)
|   ├── db.js                 the only file that touches IndexedDB
|   ├── store.js              current data, save and delete, notify screens
|   ├── main.js               starts the app, connects everything
|   ├── handlers.js           builds the button-name lookup tables
|   ├── globalActions.js      actions available on every screen
|   ├── router.js             picks the screen from the address
|   ├── charts.js             donut and bar charts
|   ├── exporter.js           JSON, CSV and Excel export; JSON import checks
|   ├── xlsx.js               builds an .xlsx file without a library
|   ├── views/                one file per screen (5 screens plus an index)
|   └── ui/                   small helpers: dom, icons, toast, theme,
install, txRow, txDialog
├── tests/run.mjs             fast tests of logic, security rules and colors
├── e2e/                      real-browser tests and a local server
├── eslint.config.js          lint rules
├── .github/workflows/ci.yml runs everything on each push
└── docs/                     flowchart.png and flowchart.svg
```

6.1 Layers

Layer	Files	Rule
Pure logic	config.js, models.js, calc.js, xlsx.js	Must not import UI, database or store code, and must not touch the browser
Storage	db.js, store.js	db.js alone talks to IndexedDB; store.js holds the current data
Presentation	views/, ui/, charts.js	Draw HTML from data; never write to the database

Layer	Files	Rule
Wiring	main.js, handlers.js, globalActions.js, router.js	Connect events to code
Delivery	index.html, css/, sw.js, manifest, vercel.json	What the browser downloads
Checks	tests/, e2e/, eslint.config.js, CI	Not part of the running app

6.2 Rules and how each is enforced

Rule	Enforced by
The pure logic files never import UI, database or store code	A unit test (machine-checked)
Every button name in the HTML has a handler, and every handler is used	A unit test (machine-checked)
sw.js lists every app file, so each one works offline	A unit test (machine-checked)
Every text and background color pair meets 4.5:1 contrast in both themes	A unit test plus a real-browser test (machine-checked)
Only db.js mentions IndexedDB	Measured by search at the time of writing; no automated test. Note that main.js also imports two lifecycle helpers from db.js
Views and UI helpers never import db.js	Measured by search at the time of writing; no automated test
Every record passes through buildTransaction() before it is saved	Convention, confirmed by reading the save and import paths; not machine-checked
User text is escaped with esc() before it enters HTML	Convention, with tests covering some outputs; not machine-checked everywhere

7. How the key features are possible

This section explains the ideas behind the features. Section 8 then shows the real code.

7.1 Saving data on the device (IndexedDB)

Browsers include a small database called IndexedDB. A website can create a database there, and the data stays on that device, in that browser, until the user clears it. The app creates one database with two object stores, which work like tables:

- **transactions**: one row per money record, keyed by a unique id, with an index on the date.
- **settings**: key and value pairs for theme, currency, budget, goal and the time of the last backup.

IndexedDB is asynchronous: you ask for something and the browser answers later. db.js wraps every request in a Promise, so the rest of the program can write await db.put(…) and read like ordinary code. Changes happen inside a **transaction**, which is all-or-nothing: either every change in it is saved or none is.

Because nothing is sent to a server, there is no account and no network requirement. That same fact is also the main weakness: the data lives in one browser on one device and can be lost (section 11).

7.2 Working offline and installing like an app (PWA)

A Progressive Web App (PWA) is a website with three extra ingredients, all present here.

1. **A manifest** (manifest.webmanifest): a small JSON file giving the app's name, colors and icons (192 and 512 pixels, plus a maskable icon), and display: standalone, which hides the browser's address bar once installed. *Measured:* a browser test fetches the manifest, checks these fields, and checks that every icon file exists.
2. **A service worker** (sw.js): a script the browser runs separately from the page. It sits between the page and the network and can answer requests from a cache. On install it saves the app files (33 cache entries *measured* in a browser test). On activation it deletes caches from older versions. On every request it uses **stale-while-revalidate**: answer from the cache immediately, and in the background fetch a fresh copy for next time.
3. **HTTPS**, which Vercel provides. Service workers only run on HTTPS or on localhost.

Result: after one visit the app opens and saves records with no internet. *Measured:* a browser test goes offline, reloads the page and adds a record. The trade-off of stale-while-revalidate is that after a new version is deployed, a user sees it one visit later. Changing CACHE_VERSION in sw.js makes the new files install as a fresh cache.

The Install button. Chrome, Edge and Android fire an event called beforeinstallprompt when the app can be installed. The app keeps that event and shows its own button, which triggers the browser's install dialog when clicked. *Measured:* a browser test fires a pretend event and clicks the button. iPhone Safari has no such event, so the app shows a hint to use Share, then Add to Home Screen. *Reasoned:* we did not test on an iPhone.

7.3 Showing the right screen (routing and rendering)

- **Routing by hash.** The part of the address after # (for example #/budget) picks the screen. It needs no server rules, which suits static hosting.
- **Views.** Each screen is an object with a render(state) function that returns HTML text, plus optional tables of handlers. main.js puts that text into the page. Adding a screen means adding one file and one line in views/index.js.
- **Event delegation.** Because screens are rebuilt after every change, individual buttons are constantly destroyed and recreated. Instead of attaching a listener to each button, the app attaches one listener to the whole document. Buttons carry a name such as data-click="add-tx", and the listener looks that name up in a Map to find the function to run.

7.4 Keeping the screen correct (store and subscribers)

store.js keeps the current data in one object, plus a set of subscribers. After any change it calls emit(), which runs every subscriber. main.js subscribes its render function, so the screen is redrawn. This is a small version of the pattern used by state libraries such as Redux. Each subscriber is wrapped in try/catch so that one failing subscriber cannot stop the others.

7.5 The money math

All math lives in calc.js. Its functions take data and return results. None of them touches the screen or the database, which is what makes them easy to test.

- **Money left** for month M = everything received minus everything spent, over all records dated up to the end of M. Unspent money therefore carries over.
- **Budget status**: ratio = spent ÷ limit. The state is over when spent is above the limit, and warning when the ratio is 0.8 or more.
- **Projection** (current month only): per day = spent ÷ day of the month. Projected spending = per day × days in the month. Projected money left = money left now − per day × days still to come.

- **Worked example.** It is October 10, you have spent 500 so far, and money left is 1,200. Per day = 500 ÷ 10 = 50. Days left = 31 − 10 = 21. Projected money left = 1,200 − 50 × 21 = 150. Projected spending for the month = 50 × 31 = 1,550.
- **Weeks** run Monday to Sunday. The function weekStart() moves a date back to its Monday, and the weekly chart groups records by that Monday.
- **Rounding.** Every sum goes through r2(), which rounds to 2 decimals, because JavaScript numbers are binary floating point (0.1 + 0.2 gives 0.30000000000000004). A unit test checks that totals such as 150.3 come out exact. Section 11 explains why whole cents would be safer still.

7.6 Charts without a library

The **donut chart** draws each category as one SVG circle with only its outline. The stroke-dasharray setting draws a visible part of the circle (arc length = share × circumference) followed by a gap for the rest. stroke-dashoffset slides each arc to where the previous one ended, and rotating the group by −90 degrees makes the first arc start at the top. The **bar chart** scales each bar's height against the largest value. Both charts give every element a title for hover text, and the whole chart a role and a label for screen readers. *Reasoned:* we did not test with a screen reader.

7.7 Exports

- **CSV** is plain text, one record per row. It starts with a byte-order mark (BOM) so Excel reads symbols such as ₱ correctly. Fields containing commas or quotes are wrapped in quotes. A text cell beginning with =, +, @ or - gets a leading apostrophe so a spreadsheet cannot run it as a formula (called formula injection).
- **Excel (.xlsx)** is secretly a ZIP archive of small XML files. xlsx.js writes those XML files and packs them with the ZIP "stored" method (no compression), adding the CRC-32 checksum that ZIP requires. *Measured:* the output opens in a spreadsheet reader (the Python library openpyxl), and a browser test checks that the download starts with PK, the ZIP signature. Limits: one sheet and no formatting.
- **JSON** is the full backup and the only format that can be imported.

7.8 Importing safely

A file chosen by the user is untrusted input. parseBackup() (1) parses the JSON inside a try/catch, (2) checks an app tag and a version number, (3) runs every record through the same validator the form uses and skips bad records, reporting how many were skipped,

and (4) cleans the settings. **Merge** keeps existing records and adds or overwrites by id. **Replace** swaps the records in one database transaction.

7.9 Themes and readable colors

A theme is one attribute on the html element: data-theme is dark or light. The CSS defines each color as a variable (such as --text and --surface), and the light theme redefines them. Switching themes changes the attribute only.

Readability is measured with the WCAG contrast ratio, (L1 + 0.05) ÷ (L2 + 0.05), where L is a color's relative luminance and L1 belongs to the lighter color. Level AA asks for 4.5:1 for normal-size text. *Measured:* the first light theme used bright Google colors with white text and failed (3.06:1 on green, 3.92:1 on red, 3.56:1 on blue). The final version uses darker shades of the same color families (#188038, #C5221F, #1A73E8). A unit test now checks every text and background pair in both themes, and a browser test measures the colors actually painted on every screen. Contrast is only one part of accessibility (section 11).

8. Significant code, explained

Every excerpt below was copied from the project files. Long functions are shown whole or marked as shortened.

8.1 Validation: buildTransaction() in js/models.js

```
export function buildTransaction(input, existing = null) {
  const type = input.type === 'income' ? 'income' : input.type === 'expense'
? 'expense' : null;
  if (!type) return { ok: false, error: 'Choose Spent or Received.' };

  const amount = Math.round(Number(input.amount) * 100) / 100; // 2 decimal
places
  if (!Number.isFinite(amount) || amount <= 0) return { ok: false, error:
'Enter an amount above 0.' };
  if (amount > 1e9) return { ok: false, error: 'That amount is too large.' };

  const date = String(input.date || '');
  if (!isValidDate(date)) return { ok: false, error: 'Pick a valid date.' };

  // Unknown category? Fall back to the last one ("Other") instead of
failing.
  const list = categoriesFor(type);
```

```
  const category = list.some((c) => c.id === input.category) ? input.category
: list[list.length - 1].id;

  return {
    ok: true,
    value: {
      id: existing?.id || (typeof input.id === 'string' && input.id) ||
newId(),
      type, amount, category, date,
      note: String(input.note || '').trim().slice(0, 80),
      createdAt: existing?.createdAt || cleanTimestamp(input.createdAt) ||
Date.now(),
    },
  };
}
```

What it does. It turns any input into a clean record, or explains why it cannot. It returns an object with ok set to true or false instead of throwing an error, so the caller decides how to show the message. Number.isFinite rejects NaN and Infinity, which a plain comparison would let through. The date check (isValidDate, not shown) rejects impossible dates such as February 30. The note is trimmed and cut to 80 characters.

Why it matters. This is the single gate for all data. The form uses it, a backup file import uses it, and records read back from the database use it. Bad data therefore has no other way in. **Trade-off:** an unknown category is silently changed to Other instead of rejected, which is friendly but alters the input.

8.2 Talking to the database: run() in js/db.js

```
async function run(storeName, mode, action) {
  const db = await openDB();
  return new Promise((resolve, reject) => {
    const tx = db.transaction(storeName, mode);
    const request = action(tx.objectStore(storeName));
    tx.oncomplete = () => resolve(request?.result);
    tx.onerror = () => reject(tx.error);
    tx.onabort = () => reject(tx.error);
  });
}
```

And, from openDB() in the same file:

```
  dbPromise = attempt;
  // If opening fails, forget it so the next call can try again (instead of
failing forever).
  attempt.catch(() => { if (dbPromise === attempt) dbPromise = null; });
  return attempt;
```

What it does. run() is the one doorway to the database. It opens a transaction, runs the requested action, and finishes the Promise only when the transaction completes, meaning the data is really stored. Every other database function (getAll, put, remove and so on) is a one-line call to run(). openDB() remembers the opened database so it opens only once, but if opening fails, it forgets the failure so the next call can try again.

Why it matters. The forgetting step fixes a real defect found in review: before the fix, one failed open was remembered and every later call failed until the page was reloaded. *Measured:* a test with a fake IndexedDB simulates a failed open, a blocked upgrade and a late success, and confirms recovery. It runs in Node, because the code only touches the global indexedDB when it is called.

8.3 All-or-nothing replace: replaceAll() in js/db.js

```
// Wipe a store and write new rows in ONE transaction: if anything fails,
// the old data is kept (used by "Replace" import so nothing is lost
halfway).
export const replaceAll = (store, rows) =>
  run(store, 'readwrite', (s) => { s.clear(); rows.forEach((r) => s.put(r));
});
```

What it does. It clears the store and writes the new rows inside a single transaction. If any write fails, the browser cancels the whole transaction and the old data stays.

Why it matters. The earlier version cleared first and wrote second, as two transactions. A failure between them would have lost everything. **Remaining limit:** the records are replaced atomically, but the settings (budget, goal) are written in a second step, because run() works on one object store at a time.

8.4 Saving and notifying: js/store.js

```
function emit() {
  // try/catch: if one listener crashes, the others still run.
  listeners.forEach((fn) => { try { fn(state); } catch (e) {
console.error(e); } });
}

export async function saveTransaction(input, id = null) {
  const existing = id ? state.transactions.find((t) => t.id === id) : null;
  const result = buildTransaction(input, existing);
  if (!result.ok) return result;
  await db.put('transactions', result.value);
  state.transactions = state.transactions.filter((t) => t.id !==
result.value.id);
  state.transactions.push(result.value);
  sortTx();
  emit();
  return { ok: true };
}
```

What it does. It follows a strict order: validate, write to the database, update the copy in memory, then notify the screens. If the database write throws an error, the code stops before the in-memory data changes, so memory never holds something the database refused. Editing works through the same path: the old record with the same id is filtered out and the new one added.

Trade-offs. All records are loaded into memory at startup, and the whole list is sorted again after every save. That is fine for a personal tracker and measured as linear in the amount of data (section 9), but it is not how you would build an app for millions of records.

8.5 The math: js/calc.js

```
export function totals(txs) {
  let income = 0, expense = 0;
  for (const t of txs) {
    if (t.type === 'income') income += t.amount;
    else expense += t.amount;
  }
  return { income: r2(income), expense: r2(expense), net: r2(income -
expense) };
}
```

```
export function balanceUpTo(txs, ym) {
  return totals(txs.filter((t) => monthOf(t.date) <= ym)).net;
}

export function projection(txs, ym, today, limit) {
  const thisMonth = monthOf(today);
  if (ym !== thisMonth) return { mode: ym < thisMonth ? 'past' : 'future' };

  const spent = totals(inMonth(txs, ym)).expense;
  const day = Number(today.slice(8, 10));
  const dim = daysInMonth(ym);
  const perDay = spent / day;
  const daysLeft = dim - day;
  return {
    mode: 'now',
    perDay: r2(perDay),
    daysLeft,
    projectedSpend: r2(perDay * dim),
    projectedBalance: r2(balanceUpTo(txs, ym) - perDay * daysLeft),
    overBudget: Boolean(limit) && perDay * dim > limit, // on pace to break
the limit?
  };
}
```

What it does. totals() adds up received and spent. balanceUpTo() compares months as text: because months are written as 2026-10, ordinary text comparison puts them in the correct order, so no date arithmetic is needed. projection() applies the formula from section 7.5 and returns only a mode (past or future) for other months, because a projection only makes sense for the current one.

Why it matters. These functions are pure: the same input always gives the same output, with no screen or database involved. That is why a test can check them in milliseconds. **Limits of the model:** the projection assumes spending is spread evenly across days. Real spending is lumpy (rent, a big purchase), so early in the month the projection can be far off. The day-of-month division also means that on day 1 a single large purchase is multiplied by 31.

8.6 One listener for every button: js/handlers.js and js/main.js

```
export function collectHandlers(views, kind, base = {}) {
  const map = new Map(Object.entries(base));
  for (const view of views) {
    for (const [name, fn] of Object.entries(view[kind] || {})) {
      // Two screens using the same name would silently replace each other.
      if (map.has(name)) console.warn(`Duplicate ${kind} handler "${name}" in
view "${view.id}"`);
      map.set(name, fn);
    }
  }
  return map;
}
```

```
function dispatch(attr, map, selector) {
  return async (e) => {
    const el = e.target.closest(selector);
    if (!el) return;
    if (attr === 'submit') e.preventDefault(); // an unknown form must never
reload the page
    const handler = map.get(el.dataset[attr]);
    if (!handler) {
      // A typo in data-click="..." would otherwise fail silently.
      console.warn(`No handler registered for
data-${attr}="${el.dataset[attr]}". Check the spelling in the view and in
views/*.js or globalActions.js.`);
      return;
    }
    try {
      await handler(el, { state: getState(), refresh: render }, e);
    } catch (err) {
      console.error(err);
      toast('Something went wrong. Please try again.', 'error');
    }
  };
}
```

What it does. collectHandlers() merges the handler tables from every screen into one Map and warns if two screens use the same name. dispatch() is the single listener: closest() walks up from the clicked element to the nearest one carrying the attribute, so

clicking an icon inside a button still works. It looks up the name, runs the function with the element, the current state and a refresh function, and shows an error toast if the function fails.

Why it matters. A Map is used instead of a plain object because a plain object already contains names such as constructor and toString, so a lookup could find something nobody wrote. Calling preventDefault for forms first means a form with an unknown name cannot reload the page. *Measured:* a unit test reads every screen's HTML and fails if a button name has no handler, or a handler has no button; we also changed one name on purpose to prove the test fails.

Limit. Names are plain strings. A typo is not caught while writing the code, only by that test or by the console warning when someone clicks.

8.7 Escaping user text: js/ui/dom.js

```
export const esc = (s) =>
  String(s ?? '').replace(/[&<>"']/g, (c) => ({ '&': '&amp;', '<': '&lt;',
'>': '&gt;', '"': '&quot;', "'": '&#39;' }[c]));

export function money(n, currency) {
  const abs = Math.abs(n).toLocaleString('en-US', { minimumFractionDigits: 2,
maximumFractionDigits: 2 });
  return `${n < 0 ? '-' : ''}${currency}${abs}`;
}
```

What it does. esc() replaces the five characters that have special meaning in HTML with harmless text versions. A note typed as an HTML tag is shown as text instead of being run. money() always formats with two decimals, a thousands separator and the chosen symbol, and puts the minus sign before the symbol.

Why it matters. The screens are built as text and inserted with innerHTML. Without esc(), a note such as an image tag with an onerror attribute would run as code. This attack is called cross-site scripting (XSS). A second layer is the Content-Security-Policy header in vercel.json, which tells the browser to run scripts only from the app's own site and so blocks inline handlers. *Measured:* tests feed malicious values to the screens and check that no tag is injected, and the browser tour runs under the real header with no violations.

Limit. Safety depends on every developer remembering esc() in every new template. Nothing automatically flags a missing call. A template library that escapes by default would remove that risk.

8.8 The donut chart: js/charts.js (shortened)

```
export function donut(items, currency) {
  const total = items.reduce((n, i) => n + i.total, 0);
  if (!total) return '';
  const R = 52, C = 2 * Math.PI * R; // radius and circle length
  let offset = 0;
  const arcs = items.map((i) => {
    const len = (i.total / total) * C;
    const arc = `<circle cx="70" cy="70" r="${R}" fill="none"
stroke="${i.color}" stroke-width="20"
      stroke-dasharray="${len} ${C - len}" stroke-dashoffset="${-offset}">
<title>${esc(i.label)}: ${esc(money(i.total, currency))}</title></circle>`;
    offset += len;
    return arc;
  }).join('');
  // ... the svg wrapper, rotated -90 degrees, and the center label follow
}
```

What it does. C is the length of the circle's outline. Each category's arc is its share of the total times C. The dash pattern shows that length and hides the rest, and offset moves each arc along the circle by the total length already drawn.

Why it matters. It shows how a chart can be built from basic geometry with no library, so it works offline and adds nothing to download. **Limit:** it only supports what we wrote. There is no animation and no click-to-filter.

8.9 The offline engine: the fetch handler in sw.js

```
self.addEventListener('fetch', (event) => {
  const req = event.request;
  if (req.method !== 'GET' || new URL(req.url).origin !== location.origin)
return;

  event.respondWith(
    caches.open(CACHE).then(async (cache) => {
      const cached = await cache.match(req, { ignoreSearch: true });
      const network = fetch(req).then((res) => {
        if (res.ok) cache.put(req, res.clone());
        return res;
      }).catch(() => null);
      // Offline + not cached + page navigation → fall back to the app shell.
      return cached || (await network) || (req.mode === 'navigate' ?
cache.match('index.html') : Response.error());
    })
  );
});
```

What it does. It ignores anything that is not a same-site GET request. For the rest it looks in the cache and starts a network request at the same time. If the network answers successfully it updates the cache. The user gets the cached copy if there is one, otherwise the network answer, otherwise (for a page load) the saved index.html.

Why it matters. This is why the app opens with no internet. *Measured:* a browser test goes offline, reloads, and adds a record.

Limits. (1) A network request is made even when the cache answers, which uses a little data. (2) Files refresh one at a time, so in principle a visitor could briefly run an old file beside a newer one right after a deployment. *Reasoned:* we did not observe this, but the design allows it, and the CACHE_VERSION change is the mitigation. (3) The list of files to pre-cache is written by hand, so a unit test checks that every file in js/ and css/ is listed.

8.10 Safe exports and imports: js/exporter.js

```
function csvCell(v) {
  let s = String(v);
  if (/^[=+@-]/.test(s) && typeof v === 'string') s = "'" + s; // stops
"formula injection" in Excel
  return /[",\n]/.test(s) ? `"${s.replace(/"/g, '""')}"` : s;
}
```

```
export function parseBackup(text) {
  let data;
  try { data = JSON.parse(text); } catch { return { ok: false, error: 'That
file is not valid JSON.' }; }
  if (!data || data.app !== APP_FILE_TAG ||
!Array.isArray(data.transactions)) {
    return { ok: false, error: 'This is not a Spending Tracker backup file.'
};
  }
  if (Number(data.version) > FILE_VERSION) {
    return { ok: false, error: 'This backup is from a newer version of the
app.' };
  }
  const transactions = [];
  // ... every record is then passed through buildTransaction(); bad ones are
counted and skipped
```

What it does. csvCell() makes one value safe for a CSV file: text that could be read as a spreadsheet formula gets a leading apostrophe, and values containing commas, quotes or line breaks are wrapped in quotes with inner quotes doubled. parseBackup() checks, in order: is it JSON, is it our file, is it from a version we understand, and only then looks at records.

Why it matters. A file from outside is never trusted. Failures return a clear message instead of crashing, and one bad record does not block the rest.

Limits. The formula guard also changes harmless notes that start with a minus or plus sign, such as -5 refund. It does not cover text that begins with a tab or carriage return, which some security guidance also lists. A backup is a plain text file with no encryption or signature, so anyone who has it can read it.

8.11 Building an Excel file by hand: js/xlsx.js (excerpt)

```
function crc32(bytes) {
  let c = 0xffffffff;
  for (const b of bytes) c = CRC_TABLE[(c ^ b) & 0xff] ^ (c >>> 8);
  return (c ^ 0xffffffff) >>> 0;
}

function makeHeader(layout, totalBytes, values) {
  const view = new DataView(new ArrayBuffer(totalBytes));
  for (const [field, [pos, bytes]] of Object.entries(layout)) {
    const value = values[field] ?? 0;
    if (bytes === 4) view.setUint32(pos, value, true);
    else view.setUint16(pos, value, true);
  }
  return new Uint8Array(view.buffer);
}
```

What it does. A ZIP file is a series of small binary headers and the file contents. makeHeader() writes a header from named fields (position and size in bytes), and crc32() computes the checksum each stored file needs. CRC_TABLE, not shown, is a lookup table built once.

Why it matters. It shows what a file format really is: bytes in an agreed layout. An earlier version used unlabeled number arrays that nobody could read. The review replaced them with named fields. *Measured:* the output opens correctly in a spreadsheet reader.

Limit. It writes the minimum: one sheet, text and numbers only. For formatting, formulas or large files, a library is the better tool.

8.12 Two themes from one set of variables: css/styles.css (excerpt)

```
:root {
  --bg: #1b1e1c;          /* page */
  --surface: #242825;     /* cards */
  --text: #e8ece9;
  --muted: #a0aba3;
  --accent: #2a7d4b;      /* forest green: main buttons */
  --accent-text: #6cc790; /* green used as text/links on dark */
}
html[data-theme="light"] {
  --bg: #ffffff; --surface: #ffffff; --surface-2: #f1f3f4; --border: #dadce0;
  --text: #202124; --muted: #5f6368;
  --accent: #1a73e8; --accent-text: #1967d2; --on-accent: #fff;
}
```

What it does. Every rule in the stylesheet uses variables such as var(--surface). The first block is the dark theme and the default. When the html element has data-theme set to light, the second block overrides the same names.

Why it matters. A color is defined once, so changing the palette means editing a few lines. The contrast test reads these same lines, so the palette and its check cannot drift apart. **Limit:** a theme chosen by the user is stored in the database, so the very first paint after a reload can briefly show the default theme before the saved choice loads.

8.13 The Install button: js/ui/install.js

```
export function initInstall() {
  if (isStandalone()) return; // already installed → keep buttons hidden

  window.addEventListener('beforeinstallprompt', (e) => {
    e.preventDefault();          // stop the browser's own mini-bar
    deferredPrompt = e;          // keep it so OUR button can trigger it
later
    root.dataset.installable = 'true';
  });
  window.addEventListener('appinstalled', () => {
    deferredPrompt = null;
    delete root.dataset.installable;
    toast('App installed');
  });
  if (isIOS()) root.dataset.installable = 'true';
}
```

What it does. The browser decides when the app is installable and announces it with an event. The code cancels the browser's own banner, keeps the event, and marks the page as installable, which makes every Install button visible through a CSS rule. Clicking the button later replays the saved event.

Limits. The browser, not the app, decides whether the event fires, so the button may never appear (for example in browsers without install support). On iPhone the button appears but can only show instructions.

8.14 A test that proves a fix: tests/run.mjs (shortened)

```
await test('FIX 4: createdAt must be a finite positive number', () => {
  for (const bad of [Infinity, -Infinity, NaN, -5, 0, 'abc', null, undefined,
{}]) {
    const t = buildTransaction({ type: 'expense', amount: 5, date: '2026-10-
01', createdAt: bad }).value;
    assert.ok(Number.isFinite(t.createdAt) && t.createdAt > 0, `createdAt
${String(bad)} was cleaned`);
  }
  // ... a second check feeds the same attack inside a backup file
});
```

What it does. It feeds nine bad values to the validator and asserts that each comes out as a usable number.

Why it matters. A test only has value if it fails when the bug returns. We checked this with **mutation testing** done by hand: change the code back to the bad version and confirm the test goes red. Of 10 deliberate breakages, 9 were caught at first; the one that was missed led to a stronger test, after which two further variants of it were caught as well.

9. Testing and verification

9.1 Four levels of checking

Level	Tool and location	What it checks	Result
Unit tests	Node 22 built-in assert, tests/run.mjs (296 lines)	Validation, the math, import checks, the Excel writer, security rules, color contrast, button wiring	20 test groups pass (Measured)
Real-browser tests	Playwright 1.56 with Chromium, e2e/ (about 335 lines)	The app used the way a person uses it, at a desktop-sized and a phone-sized screen	19 tests at two sizes: 38 pass (Measured)
Lint	ESLint, eslint.config.js	Unused variables, hidden built-in names, typos	Never run, because the build environment had no network. A substitute check with the TypeScript compiler found 0 unused variables or imports and 0 undefined names (Partly measured)

Level	Tool and location	What it checks	Result
Continuous integration	GitHub Actions, .github/workflows/ci.yml	Runs the three levels above on every push	Written, never run on GitHub (Unverified)

9.2 What the browser tests do

- Check the first-run screen and that dark theme is the default.
- Add allowance and spending, and check the totals; reload and check the records survived.
- Reject an invalid amount and keep the form open.
- Set a budget and check the 90% warning and the over-budget alert.
- Edit and delete a record; switch theme and check it is remembered.
- Export a JSON backup, delete everything, import it back, and check the numbers return; reject a file that is not valid JSON.
- Download the CSV and Excel files and check their content and type.
- Visit every screen and check that no error appears in the console, which also proves the security policy blocks nothing the app needs.
- Check the manifest and its icons, the security headers, and that the service worker file is never cached.
- Fire a pretend install event and click the Install button.
- Go offline after the first visit, reload, and add a record.
- Check that no screen is wider than the device (no sideways page scrolling).
- In both themes, on every screen and in the add-record form, measure the contrast of the colors actually painted, and check that every control has an accessible name.

9.3 Measured performance

These timings come from running the screen-building functions in Node with generated data. They measure building the HTML text only, not the browser's layout and painting, so a real phone will be slower.

- Home screen build at 50,000 records: about 32 ms.
- Quadrupling the data from 50,000 to 200,000 records roughly doubled the time, which is consistent with linear growth (no quadratic loop).

- The Records screen draws every row of the selected month with no paging: about 470 ms at roughly 4,000 rows in one month and about 2 seconds at roughly 16,000. A student is unlikely to reach those numbers, but the screen has no upper limit.

9.4 What the tests cannot show

- Whether the app is pleasant and clear to use. The visual design was not reviewed by a person on a real screen during the build.
- Behavior in Safari, Firefox, on an iPhone, or on real Android hardware.
- Use with a screen reader, or a full keyboard-only walkthrough.
- How the app behaves after weeks or months of real data.
- Mistakes that the AI author made in both the code and the tests in the same way (section 5.5).

10. Security and privacy

10.1 Privacy model

The app has no server, no account, no analytics and no third-party scripts. Data is stored in the browser on the user's own device and leaves it only when the user exports a file. *Reasoned:* the security policy in vercel.json limits network connections to the app's own site, and a code search found no calls to other sites. We did not inspect network traffic in a real browser session beyond the automated tests.

10.2 Threats considered

Threat	Defense	Evidence
Text typed or imported that tries to run as code (XSS)	All typed text goes through esc(); inputs are validated; the security policy blocks scripts from other sources	Measured: tests inject malicious values and check the output; browser tour runs under the real header
A malicious or damaged backup file	parseBackup() checks the format and every record, skips bad ones, and the file size is limited to 5 MB	Measured for validation; the size limit was read in code only (Reasoned)

Threat	Defense	Evidence
A spreadsheet formula hidden in a note (CSV injection)	A leading apostrophe on risky text	Reasoned: no dedicated test
Scripts loaded from other websites	The policy allows scripts only from the app's own site, and the app uses no CDN	Measured: a test checks the header and that index.html has no inline scripts or external files
Tricks in the local development server (such as asking for files outside the project)	The server refuses paths outside the project and hidden folders	Measured by hand with requests; development only
Someone with access to the device reads the data	Not defended	Gap: see 10.3

10.3 What is not protected

- **No login and no encryption.** Anyone who can open the browser profile can read the data. A shared computer is a risk.
- **Backups are plain text.** A JSON, CSV or Excel export can be read by anyone who has the file.
- **No security audit.** The checks above are the builder's own. No independent person has tested the app for weaknesses.
- **Browser extensions and a compromised device** can read or change anything the page can.

11. Limitations and why

Each item says what the limitation is, why it exists, how much it matters, and what would fix it. The first group matters most to real users.

11.1 Data and storage

Data lives in one browser on one device. *Why:* the brief asked for a client-side app with IndexedDB, so there is no server to hold a shared copy. *Impact:* there is no sync between phone and laptop. Clearing site data, switching browsers or reinstalling the browser

empties the app. A browser can also delete stored data when the device is short of space. The app asks the browser to protect the data (persist()), but the browser is free to refuse. Safari's published policy also removes site data after about a week without use for sites that are not added to the home screen (*Reasoned:* we did not test on Safari). *Mitigation present:* manual JSON backup and a reminder after 14 days. *Real fix:* an optional account and cloud storage, which would also bring a server, privacy and cost concerns.

Backups are manual. *Why:* a web page cannot silently write files to disk. *Impact:* the reminder is only a nudge; a user who ignores it can still lose data.

No login and no encryption of stored data. *Why:* out of scope for a first version, and there is no server to log in to. *Impact:* see 10.3. *Fix:* encrypt data with the browser's Web Crypto functions and a passphrase, accepting that a forgotten passphrase means lost data.

Replace import is atomic for records but not for settings. *Why:* the database helper works on one object store at a time. *Impact:* if a failure happens between the two steps, new records could sit beside the old budget. *Fix:* one transaction that spans both stores.

11.2 Money and features

Money is stored as decimal numbers rounded to two places. *Why:* it is simple and readable. *Impact:* the tests show typical sums are exact, but JavaScript numbers are binary floating point, so very long chains of arithmetic can drift. Professional systems store whole cents as integers. *Fix:* store amounts in cents and convert only when displaying.

Budgeting is by calendar month only. *Why:* it keeps the model small. *Impact:* no custom periods such as a two-week allowance cycle, no recurring allowance entries, and no category-by-week breakdown. The brief asked for spending by category and by week; the app gives both, but as separate views.

The projection is a straight-line average. *Why:* it is easy to explain and compute. *Impact:* real spending is lumpy, so a large purchase early in the month inflates the projection (section 8.5). *Fix:* a model that treats fixed bills separately.

One currency label with no conversion, and numbers are always formatted in the US style. *Why:* conversion needs live exchange rates, which need a server or an outside service.

Excel and CSV exports are basic. One sheet, no formatting, dates written as text. They cannot be imported, by design.

11.3 Design and code

The whole screen is rebuilt after every change. *Why:* it keeps the design simple and the data flow one-way (section 3.3). *Impact:* a redraw resets short-lived screen state, such as typing in a form if a redraw happens meanwhile, and the cost grows with the size of the list (section 9.3). *Fix:* paging for long lists, or a framework that updates only what changed.

Safety against XSS depends on remembering esc(). *Why:* screens are built from template strings. *Fix:* a template system that escapes by default.

No static types. *Why:* plain JavaScript was chosen for learning and for having no build step. *Impact:* a wrong field name is caught only when a test or a user hits it. The TypeScript compiler reports 373 type warnings on this code because it is not annotated. *Fix:* TypeScript, or JSDoc types checked by the compiler.

Button names are plain strings. A typo is caught by a test and by a console warning, not while typing (section 8.6).

Some rules are conventions, not machine-checked, for example that every record passes through the validator (section 6.2). A future change could break them without a test failing.

The first paint can show the default theme. *Reasoned:* the page starts in dark, and a saved light choice is applied after the data loads, so a light-theme user may see a brief dark flash. *Fix:* also remember the theme in a place that can be read before the first paint.

11.4 Offline and browser support

Updates reach users one visit late. *Why:* stale-while-revalidate trades freshness for speed (section 8.9). The hand-written file list is guarded by a test, but a missed change to CACHE_VERSION delays an update.

Mixed old and new files are possible for a moment after a deployment. *Reasoned:* not observed, but allowed by the design.

Modern browsers only. The app uses ES modules, IndexedDB, the dialog element, CSS color-mix(), structuredClone and the service worker. *Reasoned:* older browsers would not run it, and we did no compatibility testing. Firefox on desktop has no month picker, so the month is typed as text and validated.

The Install button depends on the browser. It appears only when the browser announces the app is installable.

11.5 Accessibility

Contrast and control names are checked automatically (Measured). Not covered: testing with a screen reader, a full keyboard-only walkthrough, and zoom and large-text settings. *Reasoned:* in light mode the yellow progress-bar fill on its pale track is probably below the 3:1 contrast that WCAG asks for graphics; the percentage is also given in text, which reduces the impact.

11.6 Evidence and process

- The code and the tests share one author, an AI, so they can share blind spots (section 5.5).
- Only Chromium was used for browser tests.
- ESLint, the CI workflow and the Vercel deployment have not been run.
- There has been no real-device use and no independent review or audit.

The honest summary: the app is a well-checked prototype by automated measures. It is not yet proven by real use, and the data-loss and review gaps above are the ones to close first.

12. Exercises and discussion questions

12.1 Exercises for students

#	Exercise	Files	Level	What you learn
1	Add a spending category called Pets, then run npm test	js/config.js	Easy	Why lists live in one config file: the form, charts and budget screen update by themselves
2	In the browser's developer tools, open the Application tab and find the app's IndexedDB data	none	Easy	Where the data really lives

#	Exercise	Files	Level	What you learn
3	Set a breakpoint in saveTransaction and step through validate, write, notify	js/store.js	Easy	The order of operations (section 8.4)
4	Break something on purpose: remove esc() from one template, run npm test, and read which test fails	js/ui/dom.js, a view	Medium	Why tests matter; hand-made mutation testing
5	Add a Help screen	a new file in js/views/, js/views/index.js, sw.js	Medium	The screen registry, and the test that reminds you to update the offline file list
6	Add a Payment method field to records	models.js, ui/txDialog.js, exporter.js	Medium	The validation gate; why IndexedDB needs no schema change for a new field
7	Change the backup reminder to 7 days and write a unit test for it	config.js, tests/run.mjs	Medium	Pure functions are easy to test
8	Add paging to the Records screen and measure the time before and after with 5,000 records	js/views/transactions.js	Hard	Performance and its trade-offs
9	Switch money to whole cents. Write the tests first	models.js, calc.js, ui/dom.js, exporter.js	Hard	Why professionals avoid floating point for money

#	Exercise	Files	Level	What you learn
10	Add a recurring weekly allowance	models.js, store.js, calc.js	Hard	A design choice: generate real records, or store a rule and calculate

12.2 Discussion questions for professors and classes

1. Is building screens from template strings and innerHTML acceptable at this size? What would make you switch to a framework?
2. The program validates data in one function used by every entry route. What are the advantages and risks of one gate compared with validating in each form?
3. The code and its tests were written by the same AI. What further evidence would you want before trusting it, and how would you organize an independent review?
4. Is a backup reminder an adequate answer to the risk of losing local data? What alternatives exist, and what does each cost in privacy and complexity?
5. Compare localStorage, IndexedDB and a server database for this problem in terms of privacy, durability, size limits and complexity.
6. Discuss floating-point numbers versus whole cents for money. How would you design tests to catch rounding errors?
7. The security design is layered: validation, then escaping, then a security policy. Where could an attacker still get through?
8. Beyond contrast and control names, what else would you test to call the app accessible?
9. How should a course treat a project built largely by an AI assistant? What should the student be able to explain without help?

12.3 What a student should be able to explain without help

A short oral check that confirms understanding rather than copying:

- Draw or describe the eight steps in the flowchart from memory.
- Explain why all data passes through one validation function.
- Explain the difference between localStorage and IndexedDB, and what a transaction guarantees.

- Explain what a service worker does and why stale-while-revalidate makes updates arrive one visit late.
- Explain what esc() prevents and how an attack would work without it.
- Explain what a pure function is and why calc.js is made of them.
- Name three limitations from section 11 and what would fix each.

13. Running, testing and deploying

13.1 Commands

First time only: run npm install, then npx playwright install chromium. After that:

```
npm start            local server at http://localhost:4173 with the same
security headers as Vercel
npm run lint         code spell-checker (ESLint)
npm test             fast logic, security and color tests
npm run test:e2e     real-browser tests, desktop and phone sizes
npm run check        all three of the above
```

13.2 Deploying

1. Create a GitHub repository and push the project folder to it.
2. On vercel.com choose Add New, then Project, then import the repository.
3. Leave the framework as Other, with no build command and no output directory, and deploy. vercel.json also tells Vercel to skip installing the development tools (not tested by us).
4. Every push to the main branch redeploys. To stop a broken version going live, require the CI check to pass in the GitHub branch settings.
5. Open the Vercel address in Chrome or Edge and use the Install button.

13.3 Releasing an update and changing the database

- Changed any app file: change CACHE_VERSION in sw.js, so returning users get fresh files.
- Added a file to js/ or css/: add it to the file list in sw.js. A unit test fails if you forget.
- Changed the database structure (new object store or index): raise DB_VERSION in config.js. Users with an old tab still open will see a message to close it and reload.

13.4 When something breaks

Symptom	Where to look
A button does nothing	The console: a warning says No handler registered for that name. Check the spelling in the view
Records seem missing	Check the month shown at the top of the screen; then developer tools, Application tab, IndexedDB
A new version does not appear	Application tab, Service Workers: update or unregister; check that CACHE_VERSION was changed
The Install button is missing	Application tab, Manifest; check the browser supports install and that the app is not already installed
A test fails	Read the message: the tests name the file and the rule that broke
A phone screen scrolls sideways	The no-screen-wider-than-the-device browser test points at the screen

14. Glossary

Term	Meaning
async / await	JavaScript keywords for waiting on something slow (such as the database) without freezing the page
Cache	A saved copy of files so they can be reused without downloading again
CI (continuous integration)	A robot that runs the tests on every push so mistakes are found early
CRC-32	A checksum used by ZIP files to detect damaged data
CSP (Content-Security-Policy)	A header that tells the browser which sources of scripts and files are allowed

Term	Meaning
ES module	A JavaScript file that imports and exports pieces, so a program can be split into files
Event delegation	One listener on a parent element handles events from many children
IndexedDB	A database built into the browser, storing data on the user’s device
JSON	A text format for structured data, used here for backups
Lint	Automatic checking of code for likely mistakes and style problems
Manifest	A small file describing an installable web app: name, icons, colors
Mutation testing	Deliberately breaking code to confirm that a test fails
One-way data flow	Data moves in one direction through the program, which makes behavior easier to predict
Promise	An object that stands for a result that will arrive later
Pure function	A function whose output depends only on its input and that changes nothing outside itself
PWA (Progressive Web App)	A website that can be installed and can work offline
Service worker	A script the browser runs between a page and the network, used here for offline mode
Stale-while-revalidate	Answer from the cache now, refresh the cache in the background
State	The data the program currently holds in memory
Transaction	A group of database changes that are saved together or not at all
Validation	Checking that input is acceptable before using it

Term	Meaning
WCAG	The Web Content Accessibility Guidelines; level AA asks for 4.5:1 contrast for normal text
XSS (cross-site scripting)	An attack that makes a page run attacker-written code, usually through unescaped text

Appendix A. Key measurements

Item	Value	Source
Application JavaScript	1,582 lines in js/	Line count of the project files
Stylesheet	227 lines	Line count
Unit tests	296 lines; 20 test groups pass	Node 22 run
Browser tests	19 tests at 2 screen sizes = 38 pass	Playwright 1.56, Chromium
Import graph	No circular imports; pure logic imports no UI, database or store code	Scan of import statements
Contrast before the fix (light theme)	3.06:1 green, 3.92:1 red, 3.56:1 blue, 4.05:1 active menu text	Computed from the colors
Contrast after the fix	Every checked pair is at least 4.5:1 in both themes	Unit test and real-browser check
Phone layout bug	563 px wide on a 412 px device, before the fix	Real-browser test
Mutation checks	10 deliberate breakages: 9 caught at once; the 10th led to a stronger test, then caught	Manual runs
Static-analysis substitute	0 unused variables or imports, 0 undefined names; 373 type warnings	TypeScript compiler 6.0.3

Item	Value	Source
Home screen build time	About 32 ms at 50,000 records; roughly linear growth	Node timing of HTML building only
Not measured	Real-device speed, Safari and Firefox behavior, screen-reader use, long-term use, ESLint, CI and Vercel runs	Not performed